WBSEC · WHITE-BOX SECURITY ANALYSIS

Security Analysis Report

`.../[01] Documenten/Studie/Master/Semester 1/Software Security/Assignment 1/Task 4`

Generated	2026-10-23 09:38
wbsec	v0.1.0 · prompts v6
Backends	claude:opus@high, claude:sonnet
Engines	codeql, clang, builtin, discovery
Languages	c (4)
Mode	PoC confirmation in sandbox
Duration	220 s · 35 model calls

Summary

REPORTED	POC-CONFIRMED	HUMAN REVIEW	HARDENING
1	**1**	**0**	**19**
0 suppressed	oracle + location	high-sev, low-conf	not exploitable alone

SEVERITY × CONFIDENCE

SEVERITY	THEORETICAL	PROBABLE	CONFIRMED
Critical	0	0	0
High	0	0	0
Medium	0	0	**1**
Low	0	0	0

Findings index

#	SEVERITY	CONFIDENCE	CWE	TITLE	LOCATION
1	Medium	Confirmed	CWE-770	malloc() size comes from ftell() (line 344) with no upper bound check — unbounded allocati	`common.c:348`

Findings

1. malloc() size comes from ftell() (line 344) with no upper bound check — unbounded allocati

MEDIUM 4.6 CONFIRMED CWE-770

Report

Location	`common.c:348 in read_text_file()`
Engines	builtin, discovery
CVSS v4	`CVSS:4.0/AV:L/AC:L/AT:N/PR:N/UI:P/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N`

WHAT

malloc() size comes from ftell() (line 344) with no upper bound check — unbounded allocation / memory exhaustion

SEVERITY RATIONALE

The allocation size at common.c:344 comes from ftell() on a file the program reads, so an attacker must supply or plant a large or crafted file (for example a sparse or special file) that a user or workflow then processes, giving AV:L and UI:P with no privileges or special conditions needed. Exploitation is simple, but the only effect is a failed or oversized allocation causing memory exhaustion or a crash in the process. That is a bounded, local availability impact (VA:L) with no confidentiality or integrity effect. No impact on subsequent systems is shown, and the score is approximate pending exact v4.0 calculator confirmation. Raise to AV:N and VA:H only if remote file input or persistent service-wide denial of service is demonstrated.

SOURCE -> SINK PATH

`common.c:344`	SOURCE	long size = ftell(f);
`common.c:348`	SINK	char *buf = malloc((size_t)size + 1);

TRIAGE

- claude:opus@high -> weakness (lines [344, 345, 348, 349, 351]): The allocation size comes from the file length returned by ftell at line 344. That value is used directly in malloc((size_t)size + 1) at line 348. The only guards are the negative-size check at line 345 and the NULL check at line 349, so there is no upper cap. However, malloc failure is handled cleanly, and the attacker would have to place a very large file at a path the program reads. That makes this a missing-hardening issue rather than a demonstrated exploit.
- claude:sonnet -> weakness (lines [344, 345, 348, 349]): The size comes from the file length via ftell (L344) and is passed straight to malloc((size_t)size + 1) at L348 with no upper cap, so a very large file forces a proportionally large allocation and whole-file read. Guards only reject errors (size < 0 at L345) and malloc failure (L349), which is handled cleanly by returning NULL. The size is bounded by the actual file on disk, and the path is chosen by the caller, so exploitation needs an attacker-controlled path or oversized file, which is an unusual precondition. Whether callers expose this to untrusted input is not shown.

Consensus: weakness

PROOF OF CONCEPT

Confirmed by oracle + location check

backend claude:opus@high · attempts 1 · oracle: sanitizer: ERROR: AddressSanitizer: requested; location matched against common.c:348

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>

extern char *read_text_file(const char *path, size_t *out_len);

int main(void)
{
    const char *path = "poc_huge.bin";
    /* Create a sparse file with an enormous logical size so that ftell()
       in read_text_file reports a huge value and the unbounded malloc on
       common.c:348 requests a gigantic allocation (CWE-770). */
    int fd = open(path, O_RDWR | O_CREAT | O_TRUNC, 0600);
    if (fd < 0) { perror("open"); return 1; }
    /* ~1 PiB logical size, stays sparse on disk. */
    off_t huge = (off_t)1 << 50;
```

```
    if (ftruncate(fd, huge) != 0) { perror("ftruncate"); close(fd); return 1; }
    close(fd);

    size_t len = 0;
    char *buf = read_text_file(path, &len);
    if (buf) { free(buf); }
    return 0;
}
```

Sanitiser / runtime output (c)

```
=================================================================
==1==ERROR: AddressSanitizer: requested allocation size 0x4000000000001 (0x4000000001008 after adjustments f
or alignment, red zones etc.) exceeds maximum supported size of 0x10000000000 (thread T0)
    #0 0xffff9b45b734 in __interceptor_malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0xaaaad5434a38 in read_text_file /work/common.c:348
    #2 0xaaaad5432f68 in main /work/driver.c:23
    #3 0xffff9a737740  (/lib/aarch64-linux-gnu/libc.so.6+0x27740)
    #4 0xffff9a737814 in __libc_start_main (/lib/aarch64-linux-gnu/libc.so.6+0x27814)
    #5 0xaaaad5432d2c in _start (/work/poc_bin+0x2d2c)

==1==HINT: if you don't care about these errors you may set allocator_may_return_null=1
SUMMARY: AddressSanitizer: allocation-size-too-big ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:6
9 in __interceptor_malloc
==1==ABORTING
```

REMEDIATION

Root cause: read_text_file() takes the file size from ftell(f) on line 344 and passes it directly to malloc((size_t)size + 1) on line 348 without any upper bound. Any file path the caller supplies (for example a huge input file, a sparse file, or a FIFO/device reporting a large size) makes the process allocate an attacker-controlled amount of memory, exhausting the heap or crashing the process. The (size_t)size + 1 conversion is also only safe because size was checked to be non-negative, and nothing caps it below SIZE_MAX or any sane application limit.

Fix: Add a compile-time limit such as #define MAX_TEXT_FILE_SIZE (1024L * 1024L) (choose a value matching the assignment's expected input size). Right after the if (size < 0) check on line 345, add if (size > MAX_TEXT_FILE_SIZE) { fclose(f); errno = EFBIG; return NULL; }. This guard must run before rewind(f) and malloc(), so malloc((size_t)size + 1) can only ever request up to MAX_TEXT_FILE_SIZE + 1 bytes. For robustness, first call fstat(fileno(f), &st) and reject anything where !S_ISREG(st.st_mode), then use st.st_size for the same limit check, so FIFOs and devices with unreliable ftell values are refused. Also check the result of fread: if got != (size_t)size && ferror(f), free(buf) and return NULL. That check must happen before fclose(f) is called. The existing buf[got] = '\0' and *out_len = got logic can stay unchanged.

Principle: Any size that comes from untrusted input (file length, a length field in a header, Content-Length, a user-supplied count) must be checked against an explicit, application-specific maximum before it is used to allocate memory or loop. The fact that a value is non-negative or that malloc returned non-NULL is not a resource limit. Define the limit once, enforce it at the boundary where the size first becomes known, and fail closed with a clear error.

Validate: Re-run the PoC against the patched code. For example, create a file larger than the limit with truncate -s 4G big.txt (a sparse file, no real disk usage) or dd if=/dev/zero of=big.txt bs=1m count=2048, then pass it to the program that calls read_text_file. It should now return NULL quickly with no large allocation; verify with /usr/bin/time -l that the maximum resident set size stays small. Also confirm that a file of exactly MAX_TEXT_FILE_SIZE bytes is still read correctly and that out_len matches its length, that a normal small input still works, and that passing a FIFO (mkfifo p) or /dev/zero is rejected if the S_ISREG check was added. Finally, check every caller of read_text_file to make sure it handles a NULL return without dereferencing it.

Hardening notes

Weaknesses triage found present but not exploitable on their own: they need an unusual precondition or only weaken defence in depth. No PoC was attempted.

SEVERITY	CWE	TITLE	WHERE
Low	CWE-732	A file may be created here with mode 0666, which would make it world-writable	`common.c:327`

SEVERITY	CWE	TITLE	WHERE
Low	CWE-197	Length ct_len is cast to (int) as a call argument — values above the type's range truncate	`task41dec.c:104`
Low	CWE-150	`plaintext` is written to the terminal without removing control characters or escape seque	`task41dec.c:118`
Low	CWE-427	GNU Makefile on plain cmd	`Makefile:67`
Low	CWE-197	Length strlen(	`common.c:48, common.c:49`
Low	CWE-197	Length len is cast to (int) as a call argument — values above the type's range truncate or	`common.c:263`
Low	CWE-197	Length n is cast to (int) as a call argument — values above the type's range truncate or t	`common.c:301`
Low	CWE-59	fopen(	`common.c:327`
Low	CWE-427	MSVC build picks up OpenSSL from C:\OpenSSL-Win64 or C:\vcpkg, which any local user can cr	`Makefile.msvc:121`
Low	CWE-521	password read into `password` with no minimum length or strength check	`task41dec.c:83`
Low	CWE-521	password read into `password` with no minimum length or strength check	`task41enc.c:63`
Low	CWE-226	RNG failure exits the process without wiping the message and password in main	`common.c:174`
Low	CWE-316	Passwords and the message pass through the unwiped stdio input buffer	`common.c:182`
Low	CWE-591	Password, message and key buffers are not locked in memory and core dumps stay enabled	`task41enc.c:28`
Low	CWE-591	Decryptor's password, key and plaintext are not locked in memory and core dumps stay enabl	`task41dec.c:29`
Low	CWE-316	Decrypted plaintext stays in the unwiped stdout buffer	`task41dec.c:118`
Low	CWE-521	No password length or strength policy beyond rejecting an empty password	`task41enc.c:67`
Info	CWE-693	Build does not enable stack protector, _FORTIFY_SOURCE, PIE, RELRO / BIND_NOW	`Makefile:24`
Info	CWE-693	MSVC build does not turn on Control Flow Guard or CET shadow-stack compatibility	`Makefile.msvc:91`